

Objectifs

L'objectif de ce travail pratique est d'observer le comportement de l'algorithme A* et d'évaluer, de comparer et de commenter les performances obtenues par rapport à l'algorithme de Dijkstra.

Une interface graphique, semblable à celle du premier travail pratique, vous permettra de modifier interactivement les principaux paramètres du problème : taille de la grille, choix de la source et de la destination, profil des poids des arêtes et choix de l'heuristique. Vous pourrez ainsi visualiser l'évolution de la recherche d'une solution et développer une intuition sur l'influence de chaque paramètre avant de passer à l'analyse quantitative.

Algorithme A*

Applicable à la recherche d'un plus court chemin d'une source s à une destination t , l'algorithme A* est une version modifiée de celui de Dijkstra, utilisant une heuristique pour approcher la distance restant à parcourir pour atteindre la destination.

Dans l'algorithme de Dijkstra les sommets sont traités dans l'ordre croissant de la distance les séparant de la source s , jusqu'à atteindre la destination t . Dans la variante A*, l'ordre de traitement se base sur l'estimation de la longueur totale d'un chemin de s à t passant par un sommet donné i . Cette estimation est égale à la somme de la longueur du meilleur chemin connu de s à i et de l'estimation fournie par l'heuristique de la distance séparant i de t . En traitant les sommets dans l'ordre croissant de cette somme on espère diminuer le nombre d'itérations nécessaires avant d'atteindre la destination t . Le pseudocode de l'algorithme est disponible dans le support de cours.

Selon les propriétés de l'heuristique utilisée, l'algorithme A* correspondra à une méthode exacte, fournissant toujours un itinéraire optimal, ou à une méthode heuristique fournissant souvent une bonne solution mais sans garantie d'optimalité.

Création des graphes pondérés

Afin d'observer visuellement le comportement de l'algorithme, la première étape consistera à créer un labyrinthe, à partir d'une grille carrée de n rangées par n colonnes, qui sera ensuite transformé en un graphe pondéré non orienté (par suppression de murs/ajout d'arêtes et ajout de poids). Pour cela il vous suffira de reprendre votre classe `DfsGenerator` du premier travail pratique.

Mise en œuvre de l’algorithme

Vous devez programmer la méthode A* en complétant l’implémentation de la classe **AStar** fournie. Pour les heuristiques, cinq variantes seront mises en place. Dans les descriptions qui suivent le paramètre $c_{\min}$ représente le poids minimum des arêtes du réseau (accessible par la méthode `minWeight`).

- ▷ Heuristique H_0 (DIJKSTRA) : L’estimation de la distance restant à parcourir est toujours nulle ($h_i = 0$ quel que soit le sommet i). Cette variante servira de référence car la méthode correspond alors à l’algorithme de Dijkstra.
- ▷ Heuristique H_1 (INFINITY_NORM) : L’estimation de la distance restant à parcourir est égale au plus grand écart entre les coordonnées de i et t :

$$h_i = c_{\min} \cdot \max(\Delta_x, \Delta_y) = c_{\min} \cdot \max(|x_t - x_i|, |y_t - y_i|)$$

où (x_t, y_t) correspond aux coordonnées de la case (dans la grille $n \times n$) associée au sommet destination t et (x_i, y_i) correspond aux coordonnées de la case associée au sommet i . Il s’agit de la distance associée à la norme L_∞ .

- ▷ Heuristique H_2 (EUCLIDEAN_NORM) : L’estimation de la distance restant à parcourir est égale à la distance euclidienne (associée à la norme L_2) séparant i de t arrondie à l’entier inférieur :

$$h_i = c_{\min} \left\lfloor \sqrt{\Delta_x^2 + \Delta_y^2} \right\rfloor = c_{\min} \left\lfloor \sqrt{(x_t - x_i)^2 + (y_t - y_i)^2} \right\rfloor.$$

- ▷ Heuristique H_3 (MANHATTAN) : L’estimation de la distance restant à parcourir est égale à la distance de Manhattan (associée à la norme L_1) :

$$h_i = c_{\min} (\Delta_x + \Delta_y) = c_{\min} (|x_t - x_i| + |y_t - y_i|).$$

- ▷ Heuristique H_4 (K_MANHATTAN) : L’estimation de la distance restant à parcourir est égale à K fois celle de Manhattan :

$$h_i = K \cdot c_{\min} (|x_t - x_i| + |y_t - y_i|).$$

Analyse expérimentale

Plusieurs jeux de paramètres sont à votre disposition dans la classe **Experiment** dont l’implémentation doit être complétée. Cette classe fixe la taille de la grille, la source s et la destination t . Les jeux de paramètres modifient uniquement la structure du labyrinthe et la pondération de ses arêtes. Pour chaque jeu vous générerez successivement N réseaux (N est fixé à 100 dans la classe **Experiment**).

REMARQUE. Vous pouvez modifier librement les valeurs proposées afin d’observer leur influence sur le comportement et les résultats des algorithmes (vous y êtes même fortement encouragés). Mais pour le rendu final vous devez utiliser les valeurs initiales fournies.

Comparaison des heuristiques optimistes (admissibles)

Pour chacun des N réseaux vous appliquerez les quatre premiers algorithmes (H_0 à H_3) et récolterez, pour chaque algorithme, la longueur de la solution trouvée, le nombre de sommets traités (un sommet est dit traité lorsqu’il est retiré de la queue de priorité et que ses voisins

sont parcourus afin de mettre à jour les meilleures distances connues) et le nombre de sommets appartenant à l'itinéraire optimal.

Ces mesures vous permettront de calculer, par instance, la réduction du nombre de sommets traités par rapport à H_0 (en %) et le taux d'expansion utile

$$\tau = \frac{\text{nombre de sommets de l'itinéraire optimal calculé}}{\text{nombre de sommets traités}},$$

qui vaut 1 pour un algorithme idéal qui ne traite que les sommets d'un itinéraire optimal.

Vous afficherez, pour chaque algorithme, les mesures de performance suivantes, moyennées sur les N instances :

- ▷ la longueur moyenne des solutions trouvées,
- ▷ le nombre moyen de sommets traités,
- ▷ la réduction moyenne (en %) du nombre de sommets traités par rapport à H_0 , calculée par instance puis moyennée,
- ▷ le taux d'expansion utile moyen $\bar{\tau}$, calculé par instance puis moyenné.

Une heuristique optimiste h **domine** une heuristique g si $h(i) \geq g(i)$ pour tout sommet i et, si h domine g , le nombre de sommets traités en utilisant h ne sera jamais plus grand que le nombre de sommets traités en utilisant g . Vous vérifierez empiriquement que l'ordre de dominance $H_0 \prec H_1 \prec H_2 \prec H_3$ se reflète dans les métriques observées.

REMARQUE. Vous afficherez les moyennes (pour les longueurs des solutions et les nombres de sommets traités) avec trois chiffres après la virgule.

Étude de l'heuristique H_4

L'heuristique H_4 n'est optimiste (c'est-à-dire qu'elle ne surestime jamais la distance encore à parcourir jusqu'à la destination) que pour $K \leq 1$; pour $K > 1$ elle surestime la distance restante et A^* ne garantit plus l'optimalité de la solution. Vous appliquerez H_4 à chacun des N réseaux pour les valeurs suivantes de K :

$$K \in \left\{0, \frac{1}{2}, 1, 2, 4, 6, 8\right\}.$$

Notez que $K = 0$ correspond à H_0 et $K = 1$ à H_3 , ce qui vous servira de points de référence.

Pour chaque valeur de K vous afficherez les mesures de performance suivantes, moyennées sur les N instances :

- ▷ le pourcentage de solutions optimales obtenues,
- ▷ la longueur moyenne des itinéraires trouvés,
- ▷ les erreurs relatives minimale, moyenne et maximale,
- ▷ la réduction moyenne du nombre de sommets traités par rapport à H_3 ,
- ▷ la réduction relative moyenne (en %) du nombre de sommets traités par rapport à H_3 .

La réduction du nombre de sommets traités par rapport à H_3 est définie, pour chaque instance, comme la différence (nombre de sommets traités par H_3) – (nombre de sommets traités par H_4) ; elle est positive lorsque H_4 traite moins de sommets que H_3 et négative dans le cas contraire.

REMARQUE. Combiné à une heuristique consistante, l'algorithme A^* traite chaque sommet au plus une fois (tout sommet retiré de la queue de priorité n'y sera plus jamais réintroduit).

Avec une heuristique non consistante, un sommet peut être retiré puis réintroduit plusieurs fois dans la queue de priorité. Il est alors possible que, lorsque la destination t est retirée de la queue de priorité (provoquant l'arrêt de l'algorithme), la valeur λ_t ne corresponde pas à la longueur du chemin de s à t contenu dans l'arborescence codée par les marques p actuelles. Pour obtenir une longueur correcte (qui peut être inférieure à λ_t), il est nécessaire de la calculer en remontant le chemin stocké par les marques p de t vers s .

Discussion

Vous répondrez aux questions suivantes dans le fichier des résultats joint à votre rendu :

- ▷ L'ordre de dominance $H_0 \prec H_1 \prec H_2 \prec H_3$ est-il confirmé par vos résultats ? L'heuristique H_4 avec $K = \frac{1}{2}$ s'insère-t-elle dans cette séquence ?
- ▷ Lorsque $K > 1$, H_4 n'est plus optimiste et A^* ne garantit plus l'optimalité de la solution. En vous appuyant sur vos résultats et observations, discutez le compromis entre qualité de la solution et efficacité de la recherche.
- ▷ La perte d'optimalité observée en utilisant H_4 avec $K > 1$ est-elle uniforme selon les jeux de paramètres, ou certains profils de graphe semblent-ils plus robustes à l'augmentation de K ? Avez-vous une explication intuitive de ce phénomène ?

Modalités et délais

- ▷ Le travail de programmation est à effectuer par groupe de deux, en Java, version 21 ou 25.
- ▷ L'archive contenant les sources du projet est disponible sur le site Cyberlearn du cours.
- ▷ Vous devez rendre une archive (au format **zip**) du répertoire **GroupX** (au nom ajusté selon votre groupe), et uniquement ce répertoire, contenant toutes vos sources, soigneusement documentées, ainsi qu'un fichier texte (ou markdown) contenant les statistiques sur les performances obtenues pour les jeux de paramètres fournis. Vous complèterez ce fichier avec vos réponses aux questions de la sous-section « Discussion ».
- ▷ Vous devez rendre votre travail sur Cyberlearn au plus tard le **dimanche 14 juin 2026** (avant minuit).